SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities. (BL5, BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

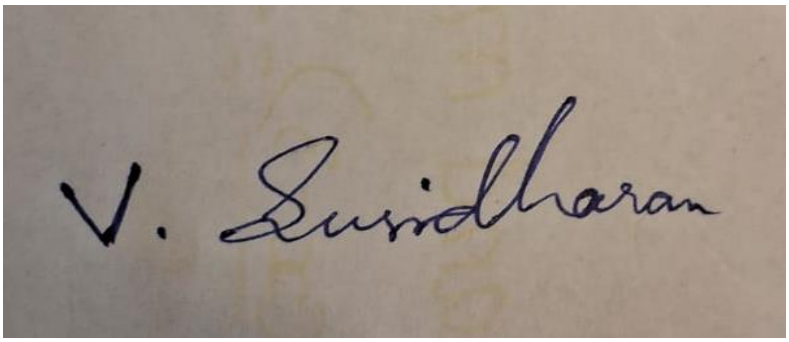
CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct:

I susidharan(192521387) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student



Question 1: IDS Deployment under Real-Time Constraints

Constraints: latency ≤ 5 ms/packet; accuracy ≥ 96%; false positive ≤ 1.5%; 150,000 packets/sec; limited shared CPU.

Design: tiered hybrid IDS

- Fast path (every packet): signature-based matching using hash-indexed lookup / Aho-Corasick multi-pattern search against known-attack signatures — average-case near O(1)-O(log n) lookup, keeping per-packet latency well under 5 ms even at 150,000 pps.
- Slow path (sampled subset only): a lightweight, statistical anomaly detector examines flow-level metadata (not full deep packet inspection) on a bounded sample of traffic, catching patterns signatures miss without processing every packet, keeping CPU load bounded.
- Parallelize the fast path across available CPU cores / NIC offload to sustain the 150,000 pps rate within the shared-CPU budget.

Trade-off analysis and justification

Pure anomaly-based detection would improve novel-attack accuracy but requires per-packet statistical modeling that is too CPU-expensive and too slow to guarantee ≤5 ms latency at this packet rate. Pure signature-based detection is fast and low-FP but blind to unknown attack patterns, capping achievable accuracy. The hybrid design meets all four constraints simultaneously: latency is bounded by the fast signature path (not the heavier anomaly path); accuracy is lifted above 96% by combining both methods; the anomaly detector is tuned conservatively (raising its alert threshold) so it doesn't push overall false positives above 1.5%; and confining expensive analysis to a sampled subset keeps CPU usage compatible with a shared-resource environment.

Question 2: Worm Containment under Network Availability Constraints

Constraints: downtime ≤ 1%; detect + contain within 2 minutes; multiple distributed locations; limited centralized control.

Design: distributed, automated micro-segmentation

- Deploy IDS/IPS sensors and local firewall enforcement at every site (edge-level, not centrally coordinated) so containment decisions are made locally and immediately — essential given limited centralized control.
- Each site's IDS watches for worm-characteristic behavior (rapid outbound connections to many hosts on a specific port/protocol) and automatically triggers local micro-segmentation — VLAN isolation or dynamic ACL push — quarantining only the affected subnet, not the whole site.
- Once a site detects the worm's signature/behavior, it broadcasts a lightweight alert (pub-sub, not requiring central approval) so every other site's firewall can pre-emptively block the same propagation vector network-wide within the 2-minute window.

Trade-off analysis and justification

Isolating an entire compromised site guarantees containment but causes excessive downtime and business disruption, violating the ≤1% downtime constraint. Relying on manual, centrally-approved containment decisions is safer against over-blocking but far too slow to meet a 2-minute detection-to-containment window across multiple locations. Automated, localized micro-segmentation resolves this: because each site acts independently and

immediately on its own detection, containment happens within 2 minutes without waiting for central coordination (satisfying the limited-centralized-control constraint), and because only the specific infected subnet is isolated rather than an entire site, total downtime stays within the ≤1% budget while unaffected segments and locations continue operating normally.

Question 3: Firewall Rule Optimization under Performance Constraints

Constraints: 10,000+ existing rules; throughput ≥ 800 Mbps; minimize rule processing time; maintain strict access control; no additional hardware.

Inefficiencies identified

- Sequential/linear rule evaluation: each packet is checked against rules in order until a match, so average lookup cost grows directly with rule count and rule position.
- Accumulated redundant, overlapping, and shadowed rules (rules that can never actually be reached because an earlier, broader rule already matches all of their traffic) inflate the rule count without adding security value.

Optimization techniques proposed

- Rule aggregation: merge multiple rules sharing the same action but differing only in IP range/port using CIDR summarization and port ranges, substantially cutting total rule count.
- Redundancy elimination: automated shadow/duplicate-rule analysis to remove rules that can never be triggered given earlier rules, without changing the effective policy.
- Prioritization/reordering: reorder rules by observed hit-count/traffic frequency so the most frequently matched rules are evaluated first, minimizing the average number of rules checked per packet — done only within equivalence classes that don't alter the effective policy, so security-critical deny rules retain correct precedence.
- More efficient matching structure: replace pure linear scanning with a decision-tree or hash-indexed rule-matching structure, moving average lookup complexity closer to O(log n) or O(1) instead of O(n).

Justification

Aggregation and redundancy elimination directly reduce the number of comparisons required per packet on average; frequency-based reordering (within policy-preserving equivalence classes) reduces the average lookup depth; and a better matching data structure reduces per-packet time complexity — together increasing achievable throughput toward and beyond 800 Mbps using only software/rule-set optimization (no new hardware), while every transformation preserves the exact same effective access-control decisions, so the strict security policy is maintained unchanged.

Question 4: IPS Deployment under Resource and Accuracy Constraints

Constraints: block ≥ 90% of known attacks; false positive ≤ 3%; 1 GB memory for signatures; 500 Mbps network speed.

Design: prioritized, memory-bounded signature strategy

- Signature selection: rank candidate signatures by real-world prevalence and severity (using current threat-intelligence feeds) and load only the highest-value signatures that fit within the 1 GB budget, using compressed/hash-based signature representations to maximize effective coverage per byte.
- Traffic filtering: apply a cheap pre-filter (protocol/port allow-list, header sanity checks) before full signature matching, reducing the volume of traffic requiring expensive deep inspection so the system keeps pace with 500 Mbps without needing more memory.
- Adaptive updates: periodically refresh the signature set, retiring rarely-triggered/low-value signatures to make room for newly relevant ones, keeping the fixed 1 GB budget always allocated to currently active threats.

Trade-off analysis and justification

Maximizing raw signature count would improve the block rate but cannot fit within 1 GB and risks including overly broad/generic patterns that raise false positives; an overly small, narrow signature set stays well within memory and FP limits but blocks fewer real attacks. Prioritizing high-prevalence, high-severity, precisely-tuned signatures resolves this tension: it concentrates the fixed memory budget on the signatures that matter most for real-world traffic (supporting ≥90% block rate), favors specificity over generic pattern matching to keep false positives ≤3%, and the pre-filter stage ensures the system still sustains 500 Mbps throughput despite the added inspection work.

Question 5: Malware Detection using Behavioral Analysis under System Constraints

Constraints: endpoint CPU ≤ 15%; detection within 10 seconds; must catch zero-day threats; limited log storage.

Design: lightweight, targeted behavioral monitoring

- Hook a small set of high-signal OS-level events only — process creation, mass/rapid file modification (e.g., ransomware-style encryption patterns), registry changes, and connections to unusual destinations — rather than full system-call tracing, keeping CPU overhead low.
- Use a lightweight weighted-scoring/heuristic engine (a handful of suspicious-behavior indicators combined against a threshold within a short time window) rather than a heavy local ML model, enabling a decision within the 10-second window on minimal CPU.
- Retain only compact behavioral summaries/hashes of flagged activity locally (not exhaustive raw logs), with older data rotated out, and forward only confirmed suspicious cases for deeper, non-time-critical central analysis — respecting limited local storage.

Trade-off analysis and justification

Comprehensive full-system tracing combined with a heavy local ML model would likely improve detection accuracy on subtle threats, but would exceed both the CPU budget and the 10-second detection window, and would generate far more log data than the storage constraint allows. An overly narrow monitoring scope would easily fit the constraints but risks missing genuine zero-day behavior. Targeting a small set of high-signal behavioral indicators strikes the balance: because it detects malicious behavior patterns (not known signatures), it can still catch zero-day/unknown threats; because it hooks only a few targeted event types and uses a lightweight scoring engine rather than heavy inference, it stays within the 15% CPU budget and meets the 10-second detection window; and retaining only compact summaries with rotation keeps it within the limited storage available.